🚀 Collections Framework Masterclass

Chapter 5 — LinkedList (Complete Internal Working)

> **Interview Importance:** ⭐⭐⭐⭐⭐

`LinkedList` is one of the most misunderstood classes in the Java Collections Framework.

Many developers believe:

> "LinkedList is always faster than ArrayList."

This is **not true**.

A 5+ years Java developer is expected to know **when LinkedList is better**, **when ArrayList is better**, and **how LinkedList works internally**.

What is LinkedList?

`LinkedList` is a **doubly linked list** implementation of the `List` interface.

Package

```
java.util.LinkedList
```

Declaration

```
public class LinkedList<E>
        extends AbstractSequentialList<E>
        implements List<E>,
                   Deque<E>,
                   Cloneable,
                   Serializable
```

Notice

Unlike ArrayList,

LinkedList implements

- List
- Queue
- Deque

That makes it much more versatile.

Position in Collection Hierarchy

```
Iterable
    ↑
Collection
    ↑
List
    ↑
LinkedList
```

Also

```
Deque
    ↑
LinkedList
```

LinkedList can behave as

- List
- Queue
- Deque
- Stack

Characteristics

✅ Maintains insertion order

✅ Allows duplicates

✅ Allows multiple null values

✅ Dynamic size

✅ Fast insertion/deletion after locating the position

❌ Slow random access

❌ Higher memory consumption

❌ Not thread-safe

Internal Structure

Unlike ArrayList,

LinkedList does NOT use an array.

It uses **Nodes**.

Each node stores

- Data
- Previous node reference
- Next node reference

Example

```
+------+-------+-------+
| Data | Prev  | Next  |
+------+-------+-------+
```

Node Class (Simplified)

Internally Java has something similar to

```
private static class Node<E> {

    E item;

    Node<E> next;

    Node<E> prev;
```

```
    Node(Node<E> prev,
         E element,
         Node<E> next) {

        this.item = element;
        this.prev = prev;
        this.next = next;
    }
}
```

Every element becomes a Node.

Memory Representation

Suppose

```
list.add("A");
list.add("B");
list.add("C");
```

Memory

```
head

↓

+---------------------------+
| Prev | A | Next ---------|
+---------------------------+
              |
              ↓
+---------------------------+
| Prev | B | Next ---------|
+---------------------------+
              |
              ↓
+---------------------------+
| Prev | C | Next = null   |
+---------------------------+

tail
```

Each node knows

- Previous node
- Next node

Hence

"Doubly Linked List"

Why Doubly Linked List?

Suppose Java used

```
Singly Linked List
```

Backward traversal would not be possible.

Removing a node would require finding its previous node.

A doubly linked list allows

- Forward traversal
- Backward traversal
- Easier deletion

Head and Tail References

LinkedList stores

```
Node<E> first;

Node<E> last;
```

These references allow efficient insertion/removal at both ends.

Adding Elements

Example

```
LinkedList<Integer> list =
new LinkedList<>();

list.add(10);
list.add(20);
list.add(30);
```

Memory

```
10 ⇄ 20 ⇄ 30
```

New node

```
40
```

After insertion

```
10 ⇄ 20 ⇄ 30 ⇄ 40
```

Tail reference updates.

Complexity

```
O(1)
```

Accessing Elements

Suppose

```
list.get(5);
```

Can LinkedList directly jump?

No.

It must traverse.

Example

```
A ⇄ B ⇄ C ⇄ D ⇄ E ⇄ F
```

Need index 5

Traversal

```
A

↓

B

↓

C

↓

D

↓

E

↓

F
```

Complexity

```
O(n)
```

Interview Favorite ⭐⭐⭐⭐⭐

Optimization Inside LinkedList

Java is smarter.

Suppose

Need

```
list.get(98);
```

Total size

```
100
```

Java starts from

```
Tail
```

instead of Head.

Similarly

Need

```
list.get(2);
```

Starts from

```
Head
```

Algorithm

```
If

index < size/2

↓

Start from Head

Else

↓

Start from Tail
```

Interview Favorite ⭐⭐⭐⭐⭐

Insertion

Adding at beginning

```
list.addFirst(100);
```

Old

```
10 ⇄ 20 ⇄ 30
```

New

```
100 ⇄ 10 ⇄ 20 ⇄ 30
```

Only pointers change.

Complexity

```
O(1)
```

Adding at end

```
list.addLast(40);
```

Complexity

```
O(1)
```

Adding in middle

Need traversal first.

Example

```
list.add(5,100);
```

Complexity

```
Traversal

O(n)

Insertion

O(1)

Total

O(n)
```

Deletion

Remove first

```
list.removeFirst();
```

Pointers change.

Complexity

```
O(1)
```

Remove last

```
list.removeLast();
```

Complexity

```
O(1)
```

Remove middle

Need traversal.

Complexity

```
O(n)
```

Why Insertion is Faster?

ArrayList

```
A B C D
```

Insert at beginning

Need shifting

```
100 A B C D
```

All elements move.

LinkedList

```
100 → A → B → C → D
```

Only references change.

Queue Operations

LinkedList implements Deque.

Therefore

```
offer()

offerFirst()

offerLast()

poll()

pollFirst()

pollLast()

peek()

peekFirst()

peekLast()
```

are available.

Stack Operations

LinkedList can also behave as Stack.

Methods

```
push()

pop()

peek()
```

Example

```
LinkedList<Integer> stack =
new LinkedList<>();

stack.push(10);
stack.push(20);

System.out.println(stack.pop());
```

Output

```
20
```

Null Values

Allowed

```
list.add(null);
list.add(null);
```

Valid.

Duplicate Elements

Allowed.

```
list.add("Java");
list.add("Java");
```

Output

```
[Java, Java]
```

Thread Safety

Interview Favorite ⭐⭐⭐⭐⭐

Is LinkedList synchronized?

No.

It is not thread-safe.

Internal Source Code (Simplified)

Adding last

```
void linkLast(E e) {

    Node<E> l = last;

    Node<E> newNode =
        new Node<>(l, e, null);

    last = newNode;

    if(l == null)
        first = newNode;
    else
        l.next = newNode;
}
```

Only references change.

Memory Usage

Interview Favorite ⭐⭐⭐⭐

Each node stores

- Data
- Previous reference
- Next reference

Compared to ArrayList,

Memory usage is higher.

Example

```
ArrayList

A B C D
```

LinkedList

```
Prev A Next

Prev B Next

Prev C Next

Prev D Next
```

Extra references consume additional memory.

Cache Locality

ArrayList

```
A B C D E
```

Stored contiguously.

CPU cache-friendly.

LinkedList

Nodes are scattered in memory.

Poor cache locality.

Hence iteration is generally slower than ArrayList.

Interview Favorite ⭐⭐⭐⭐⭐

ArrayList vs LinkedList

Feature	ArrayList	LinkedList
Internal Structure	Dynamic Array	Doubly Linked List
Random Access	O(1)	O(n)
Insert at End	O(1)*	O(1)
Insert at Beginning	O(n)	O(1)

Feature	ArrayList	LinkedList
Remove at Beginning	O(n)	O(1)
Memory Usage	Lower	Higher
Cache Friendly	Yes	No
Implements Deque	No	Yes

*Amortized O(1)

When Should You Use LinkedList?

Use LinkedList when

- Frequent insertions/removals at the beginning or end
- Queue implementation
- Deque implementation
- Stack implementation (though `ArrayDeque` is generally preferred)
- Sequential traversal

Avoid LinkedList when

- Frequent random access
- Read-heavy applications
- Memory optimization is important

Common Mistakes

Mistake 1

Using LinkedList for random access.

```
list.get(90000);
```

Slow.

Mistake 2

Using LinkedList assuming insertion is always O(1).

Wrong.

Finding the insertion position requires traversal.

Total complexity

```
O(n)
```

Mistake 3

Using LinkedList instead of ArrayList for every List.

ArrayList is generally the better default choice for most applications.

Time Complexity

Operation	Complexity
get(index)	O(n)
set(index)	O(n)
addLast()	O(1)
addFirst()	O(1)
add(index)	O(n)
removeFirst()	O(1)
removeLast()	O(1)
remove(index)	O(n)
contains()	O(n)
iteration	O(n)

Frequently Asked Interview Questions

Q1. Why is LinkedList slower than ArrayList for reading?

Because it cannot directly access an index. It must traverse nodes one by one.

Q2. Why is LinkedList faster for insertion?

Once the target position is reached, only node references (`prev` and `next`) need to be updated. No element shifting is required.

Q3. Does LinkedList use a singly or doubly linked list?

Doubly linked list.

Q4. Why does LinkedList implement Deque?

Because it supports efficient insertion and removal from both the front and the rear.

Q5. Can LinkedList be used as a Queue?

Yes.

Methods include:

```
offer();
poll();
peek();
```

Q6. Can LinkedList be used as a Stack?

Yes.

Methods include:

```
push();
pop();
peek();
```

However, `ArrayDeque` is generally recommended for stack implementations.

Q7. Why does LinkedList consume more memory?

Each node stores the element plus two additional references (`prev` and `next`).

Q8. How does get(index) work internally?

Java chooses the shorter traversal path:

- Starts from `first` if the index is in the first half.
- Starts from `last` if the index is in the second half.

Q9. Is LinkedList thread-safe?

No.

Q10. Which is better: ArrayList or LinkedList?

There is no universal winner.

- **ArrayList** is better for random access and read-heavy workloads.
- **LinkedList** is useful for frequent insertions/removals at the ends and for queue/deque operations.

Key Takeaways

- `LinkedList` is implemented as a **doubly linked list**.
- Every element is stored in a separate `Node`.
- Nodes maintain references to both the previous and next nodes.
- Random access is O(n), while insertion/removal at the ends is O(1).
- LinkedList uses more memory than ArrayList because of additional node references.
- Poor cache locality makes iteration slower than ArrayList in many real-world scenarios.
- `LinkedList` also implements the `Deque` interface, enabling queue and stack operations.

Next Chapter

🚀 Chapter 6 – Vector & Stack (Legacy Collections Deep Dive)

We'll cover:

- History of Vector

- Vector Internal Architecture
- Capacity Increment
- Vector vs ArrayList
- Thread Safety
- Synchronization Cost
- Stack Internals
- Why Stack is Considered Legacy
- Stack vs Deque
- Real-world Usage
- Performance Analysis
- 20+ Interview Questions